

Cyberster Internship Program

Internship Report

TRACK: BLUE TEAM

NAME: MUHAMMAD RAZA

INSTRUCTOR: ABDULLAH ZIA

ROLL NUMBER: CSI-B1-121

WEEK 3: SURICATA IDS INTEGRATION WITH WAZUH

TABLE OF CONTENTS

Overview ____ 4

Days 15 and 16 — Suricata Installation and Configuration ____ 4

Conceptual Foundations ____ 4

Installation and Network Configuration ____ 5

Day 17 — Suricata Rule Writing ____ 6

Rule Engineering ____ 6

Threat Execution and Alert Verification ____ 8

Days 18 and 19 — Suricata and Wazuh Integration ____ 9

Log Forwarding and Troubleshooting ____ 9

Dashboard Verification ____ 11

Event Correlation Analysis ____ 12

Days 20 and 21 — GeoIP Blocking and Firewall Rules ____ 13

Implementing GeoIP Restrictions ____ 13

Local DNS Sinkholing ____ 15

Security Limitations and Wazuh Visibility ____ 15

Overview

Week 3 transitions the SOC environment from exclusively host-based monitoring to comprehensive network visibility and perimeter defense. While Wazuh is highly effective at tracking file modifications, log anomalies, and authentication attempts, it inherently lacks visibility into raw network traffic. To eliminate this blind spot, **Suricata**, a high-performance Network Intrusion Detection System (NIDS), was deployed.

The primary objective of this week was to install Suricata, write custom signature-based detection rules for specific threats, forward the network alerts into the central Wazuh Manager, and correlate network attacks with host-level breaches. Finally, active perimeter defense was established by deploying IPSet and IPtables to block malicious traffic originating from foreign geographic regions.

Days 15 and 16 — Suricata Installation and Configuration

Conceptual Foundations

Before deployment, it is crucial to understand Suricata's operating modes and logging mechanisms:

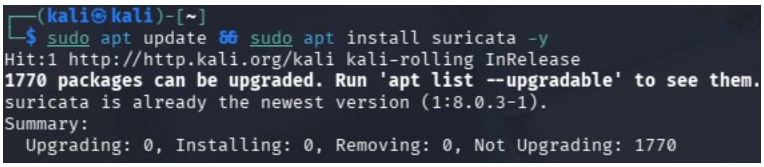
- **IDS vs. IPS:** Suricata was deployed in **Intrusion Detection System (IDS)** mode rather than Intrusion Prevention System (IPS) mode. IDS mode passively inspects copies of network traffic and generates alerts without interfering with packets. IPS mode sits inline and drops traffic, which can cause catastrophic

self-denial-of-service if a rule is overly aggressive. In a SOC, detection must be mastered before prevention is implemented.

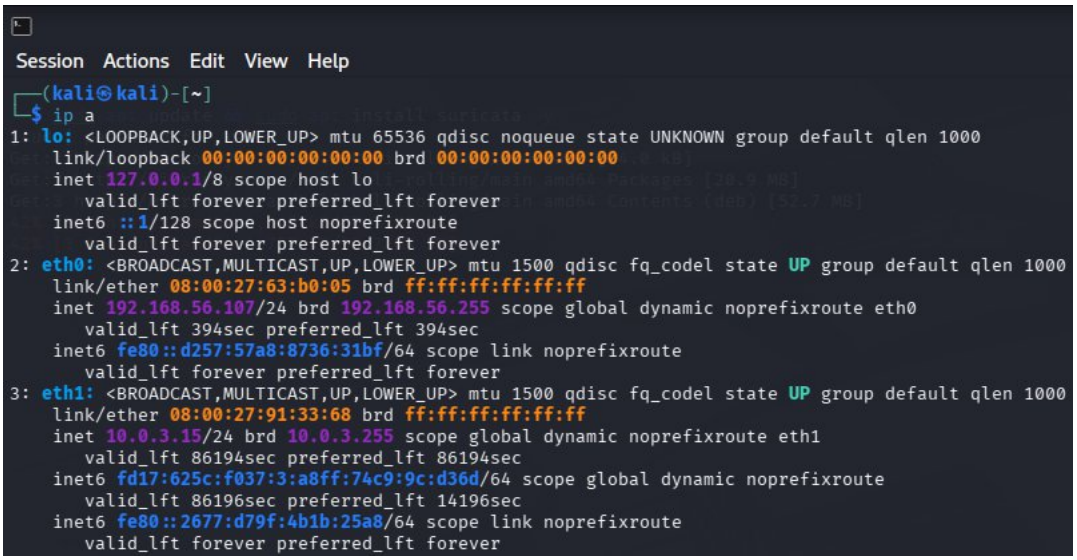
- **fast.log vs eve.json:** Suricata outputs alerts to two primary files. fast.log provides a simple, human-readable single-line summary of an alert. eve.json outputs rich, structured JSON data containing extensive metadata (HTTP headers, flow data, payloads) designed specifically for ingestion by SIEMs like Wazuh.

Installation and Network Configuration

Suricata was installed onto the Kali Linux VM to directly monitor the interface generating and receiving attack traffic.



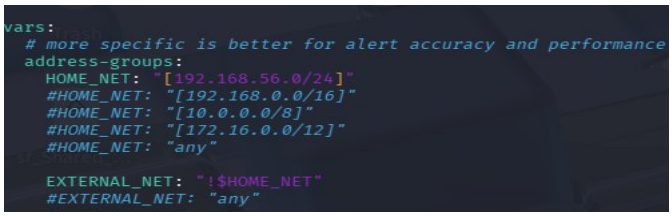
The network interfaces were verified using the ip a command to identify the correct Host-Only adapter.



The primary configuration file suricata.yaml was opened to define the network scope and capture interface.



The HOME_NET variable was explicitly configured to monitor the local lab subnet [192.168.56.0/24]. Furthermore, the af-packet capture mechanism was configured to bind to eth0. af-packet utilizes the Linux kernel's high-performance packet capture interface to drop packets directly into Suricata's analysis engine with minimal overhead.



The Suricata service was started and verified to be actively running.



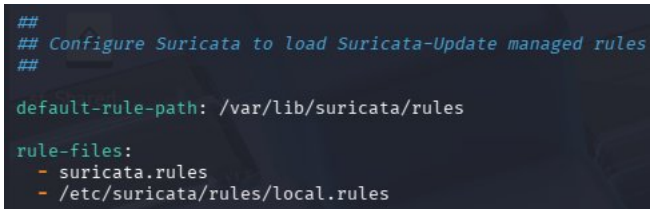
To prepare for rule testing, a continuous read (tail -f) was executed on the fast.log file.



Day 17 — Suricata Rule Writing

Rule Engineering

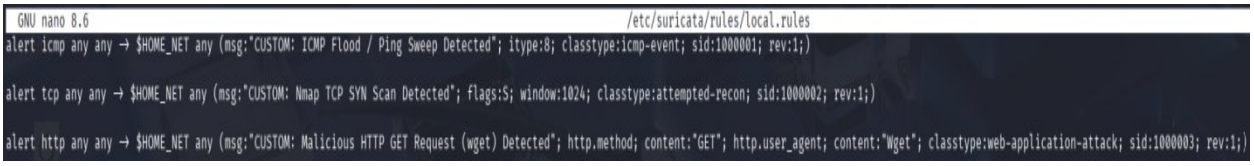
Suricata utilizes a robust syntax to perform both signature-based detection (matching explicit byte sequences) and anomaly-based detection (identifying behavioral deviations like floods or scans). The suricata.yaml file was modified to explicitly load custom signatures from a local rules file.



The local rules file was opened to author three custom detection signatures.



The rules were engineered with unique SIDs (100000X) to prevent conflicts with the community ruleset.

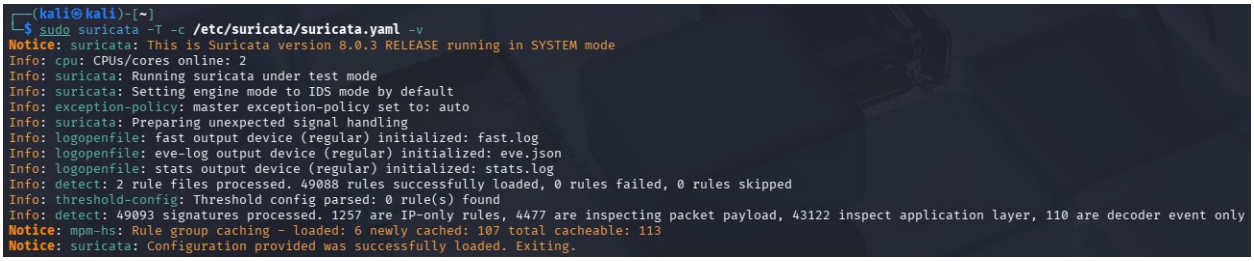


Custom Rules Breakdown

Rule Type	Threat Scenario	Rule Logic & Thresholds
ICMP Flood	Detects Denial of Service (DoS) attempts or aggressive network sweeps using rapid ping requests.	The rule inspects icmp traffic looking for itype:8 (Echo Request). A threshold parameter is essential here; without it, every legitimate ping would trigger an alert. The rule fires only if a high volume of packets from a single source exceeds the defined limit within a brief timeframe.
Nmap SYN Scan	Detects reconnaissance activity where an attacker maps open ports without fully completing the TCP handshake.	The rule looks for tcp packets sent to any port containing exclusively the SYN flag (flags:S). It is classified as attempted-recon.

Malicious HTTP	Detects Directory Traversal exploits (../../../etc/passwd) or unauthorized command-line web requests.	The rule specifically filters for the GET method and inspects the HTTP headers. It searches for content:"Wget" within the http.user_agent field, classifying it as a web-application-attack.

Following rule creation, Suricata's configuration test flag (-T) was executed to parse the custom rules and verify there were zero syntax errors.



Threat Execution and Alert Verification

To validate the rules, Suricata was started manually in the foreground, binding to both the loopback (lo) and primary (eth0) interfaces to capture local exploit attempts.

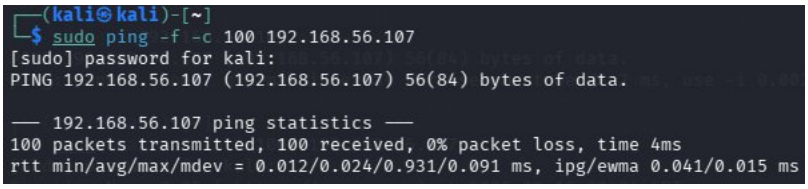


A dummy Python HTTP server was spun up to ensure web exploit payloads would successfully traverse the network rather than immediately dropping.

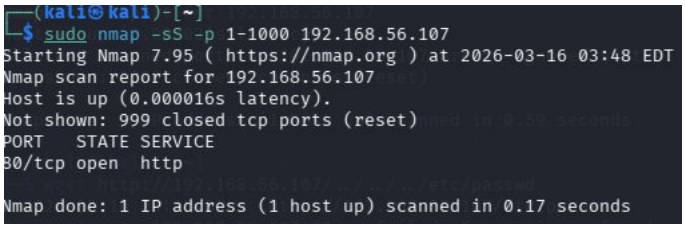


The three attacks were systematically launched against the target environment.

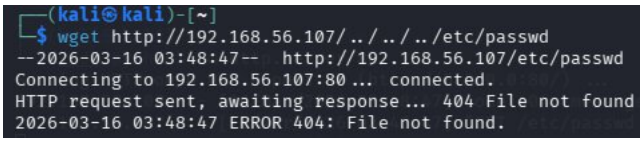
Attack 1: ICMP Flood (ping -f)



Attack 2: Nmap SYN Scan (nmap -sS)



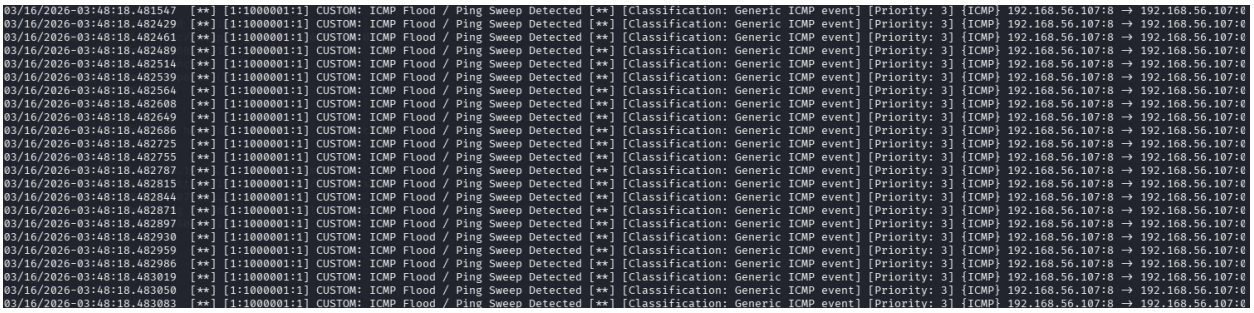
Attack 3: HTTP Exploit / Directory Traversal (wget)



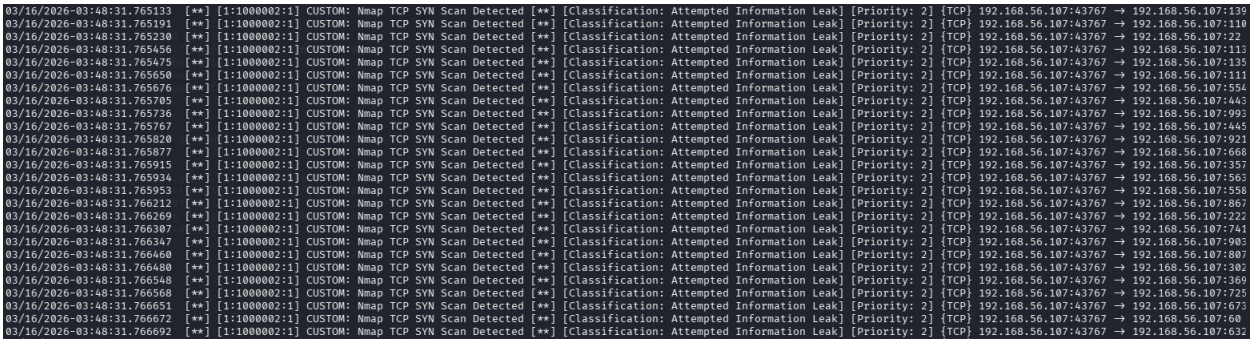
The fast.log output was queried to confirm the successful detection of all three threat scenarios.



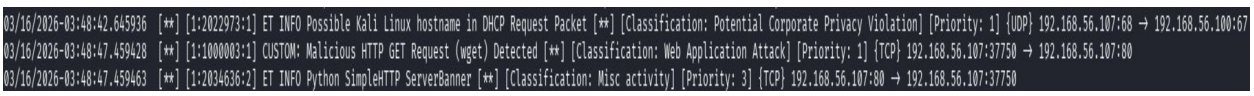
The ICMP Flood generated high-volume alerts grouped by the threshold limit.



The Nmap scan correctly triggered a barrage of attempted reconnaissance alerts.



The Malicious HTTP rule successfully caught the wget web application attack.



Days 18 and 19 — Suricata and Wazuh Integration

Log Forwarding and Troubleshooting

To unify network and host logs, the Wazuh agent on the Kali VM required configuration to read Suricata's structured JSON output. The agent configuration file was modified.

```
┌──(kali㉿kali)-[~]
└─$ sudo nano /var/ossec/etc/ossec.conf
```

A <localfile> block was appended to /var/ossec/etc/ossec.conf. Crucially, the <log_format> was explicitly defined as json. If left as default (syslog), Wazuh would fail to map the parsed fields, breaking dashboard functionality.

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

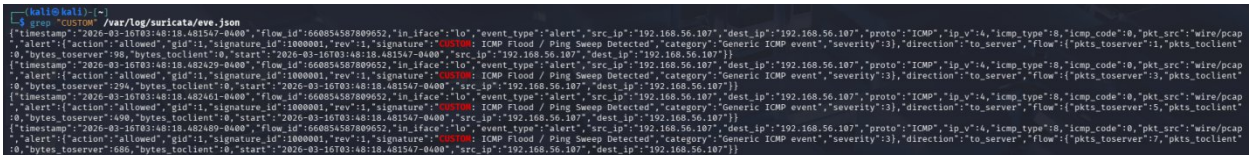
The Wazuh agent was restarted to apply the new log ingestion pipeline.

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart wazuh-agent
```

To verify Suricata was outputting properly formatted data to the pipeline source, the eve.json file was queried.



The raw JSON payload for the ICMP flood confirmed proper metadata generation.



The raw JSON payload for the Nmap scan successfully captured the flow data.



A specific HTTP exploit JSON object was extracted for debugging purposes.



Pipeline Troubleshooting: Initially, the alerts failed to appear in the Wazuh dashboard. The wazuh-logtest utility was utilized to simulate the Manager's decoding engine. An initial user error occurred by pasting the literal grep command into the utility, causing a Phase 2 decoding failure.

```
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.3
Type one log per line

grep "CUSTOM: Malicious HTTP" /var/log/suricata/eve.json | head -n 1

**Phase 1: Completed pre-decoding.
        full event: 'grep "CUSTOM: Malicious HTTP" /var/log/suricata/eve.json | head -n 1'

**Phase 2: Completed decoding.
        No decoder matched.
```

Upon pasting the raw JSON payload, the Suricata decoder successfully parsed the JSON strings in Phase 2.

```
^C[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.3
Type one log per line

{"timestamp":"2026-03-16T03:48:47.459428-0400","flow_id":1972947333558739,"in_iface":"lo","event_type":"alert","src_ip":"192.168.56.1
07","src_port":37750,"dest_ip":"192.168.56.107","dest_port":80,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","tx_id":0,"alert":{"actio
n":"allowed","gid":1,"signature_id":1000003,"rev":1,"signature":"CUSTOM: Malicious HTTP GET Request (wget) Detected","category":"Web
Application Attack","severity":1},"ts_progress":"request_complete","tc_progress":"response_body","http":{"hostname":"192.168.56.107",
"url":"/etc/passwd","http_user_agent":"Wget/1.25.0","http_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status
":404,"length":0},"app_proto":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver":411,"bytes_
toclient":391,"start":"2026-03-16T03:48:47.393826-0400","src_ip":"192.168.56.107","dest_ip":"192.168.56.107","src_port":37750,"dest_p
ort":80}}

**Phase 1: Completed pre-decoding.

**Phase 2: Completed decoding.
       name: 'json'
       alert.action: 'allowed'
       alert.category: 'Web Application Attack'
       alert.gid: '1'
       alert.rev: '1'
       alert.severity: '1'
       alert.signature: 'CUSTOM: Malicious HTTP GET Request (wget) Detected'
```

Phase 3 filtering revealed the root cause: The default Wazuh rule for Suricata (86601) assigns a level: 3 priority to these events. By default, the Wazuh dashboard only surfaces alerts of Level 4 or higher, burying the network detections as informational background noise.

```
**Phase 3: Completed filtering (rules).
        id: '86601'
        level: '3'
        description: 'Suricata: Alert - CUSTOM: Malicious HTTP GET Request (wget) Detected'
        groups: '['ids', 'suricata']'
        firedtimes: '1'
        mail: 'False'
**Alert to be generated.
```

To rectify this, a local rule override was authored on the Wazuh Manager.

```
[wazuh-user@wazuh-server ~]$ sudo nano /var/ossec/etc/rules/local_rules.xml
```

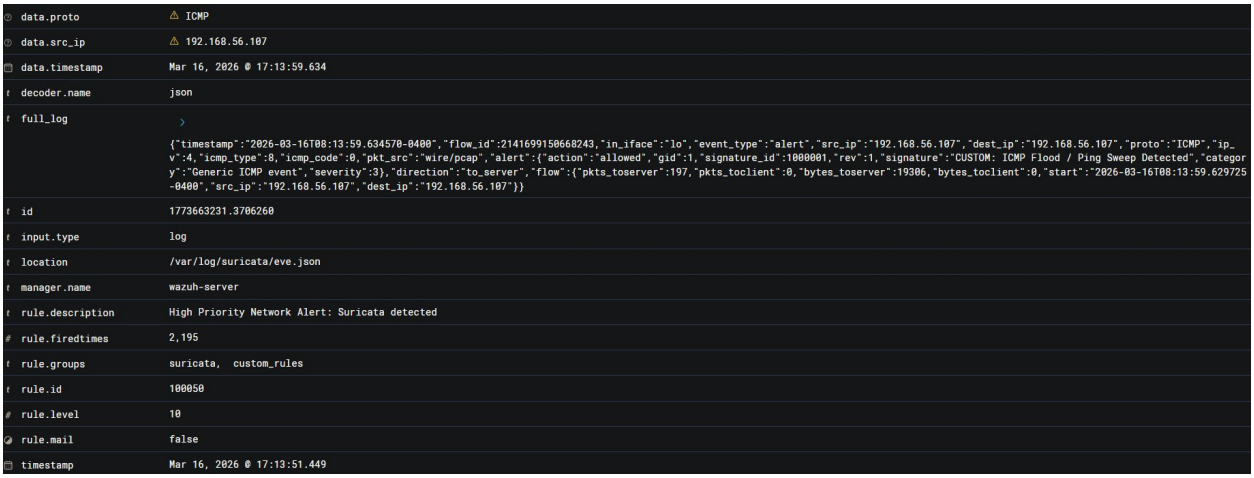
Rule 100050 was created to catch any log matching the Suricata base rule (if_sid: 86601) that contained the string "CUSTOM", automatically escalating it to a critical Level 10.

```
<group name="suricata, custom_rules,">
  <rule id="100050" level="10">
    <if_sid>86601</if_sid>
    <match>CUSTOM</match>
    <description>High Priority Network Alert: Suricata detected $(suricata.eve.alert.signature)</description>
  </rule>
</group>
```

Dashboard Verification

Following the Manager restart, the attacks were relaunched. The custom rules successfully bypassed the filtering threshold and populated the Security Events dashboard with fully parsed JSON fields.

Dashboard View: ICMP Flood



data.proto	⚠ ICMP
data.src_ip	⚠ 192.168.56.107
data.timestamp	Mar 16, 2026 @ 17:13:59.634
decoder.name	json
full_log	{"timestamp":"2026-03-16T08:13:59.634570-0400","flow_id":2141699150668243,"in_iface":"lo","event_type":"alert","src_ip":"192.168.56.107","dest_ip":"192.168.56.107","proto":"ICMP","ip_v":4,"icmp_type":8,"icmp_code":0,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"CUSTOM: ICMP Flood / Ping Sweep Detected","category":"Generic ICMP event","severity":3},"direction":"to_server","flow":{"pkts_toserver":197,"pkts_toclient":0,"bytes_toserver":19306,"bytes_toclient":0,"start":"2026-03-16T08:13:59.629725-0400","src_ip":"192.168.56.107","dest_ip":"192.168.56.107"}}
id	1773663231.3706260
input.type	log
location	/var/log/suricata/eve.json
manager.name	wazuh-server
rule.description	High Priority Network Alert: Suricata detected
rule.firedtimes	2,195
rule.groups	suricata, custom_rules
rule.id	100050
rule.level	10
rule.mail	false
timestamp	Mar 16, 2026 @ 17:13:51.449

Dashboard View: Malicious HTTP Request



data.proto	⚠ TCP
data.src_ip	⚠ 192.168.56.107
data.src_port	⚠ 38268
data.tc_progress	⚠ response_body
data.timestamp	Mar 16, 2026 @ 17:12:45.117
data.ts_progress	⚠ request_complete
data.tx_id	⚠ 0
decoder.name	json
full_log	{"timestamp":"2026-03-16T08:12:45.117068-0400","flow_id":1541499131848189,"in_iface":"lo","event_type":"alert","src_ip":"192.168.56.107","src_port":38268,"dest_ip":"192.168.56.107","dest_port":80,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","tx_id":0,"alert":{"action":"allowed","gid":1,"signature_id":1000003,"rev":1,"signature":"CUSTOM: Malicious HTTP GET Request (wget) Detected","category":"Web Application Attack","severity":1},"ts_progress":"request_complete","tc_progress":"response_body","http":{"hostname":"192.168.56.107","url":"/etc/passwd","http_user_agent":"Wget/1.25.0","http_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":404,"length":0},"app_proto":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver":411,"bytes_toclient":391,"start":"2026-03-16T08:12:45.096764-0400","src_ip":"192.168.56.107","dest_ip":"192.168.56.107","src_port":38268,"dest_port":80}}
id	1773663156.3546351
input.type	log
location	/var/log/suricata/eve.json
manager.name	wazuh-server
rule.description	High Priority Network Alert: Suricata detected
rule.firedtimes	2,096
rule.groups	suricata, custom_rules
rule.id	100050
rule.level	10

Dashboard View: Nmap TCP SYN Scan



data.pkt_src	wire/pcap
data.proto	TCP
data.src_ip	192.168.56.107
data.src_port	62859
data.timestamp	Mar 16, 2026 @ 17:10:31.994
decoder.name	json
full_log	{"timestamp":"2026-03-16T08:10:31.994566-0400","flow_id":2019828655890268,"in_iface":"lo","event_type":"alert","src_ip":"192.168.56.107","src_port":62859,"dest_ip":"192.168.56.107","dest_port":474,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000002,"rev":1,"signature":"CUSTOM: Nmap TCP SYN Scan Detected","category":"Attempted Information Leak","severity":2},"direction":"to_server","flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":58,"bytes_toclient":0,"start":"2026-03-16T08:10:31.994566-0400","src_ip":"192.168.56.107","dest_ip":"192.168.56.107","src_port":62859,"dest_port":474}}
id	1773663027.3499080
input.type	log
location	/var/log/suricata/eve.json
manager.name	wazuh-server
rule.description	High Priority Network Alert: Suricata detected
rule.firedtimes	2,095
rule.groups	suricata, custom_rules
rule.id	100050
rule.level	10
rule.mail	false

Event Correlation Analysis

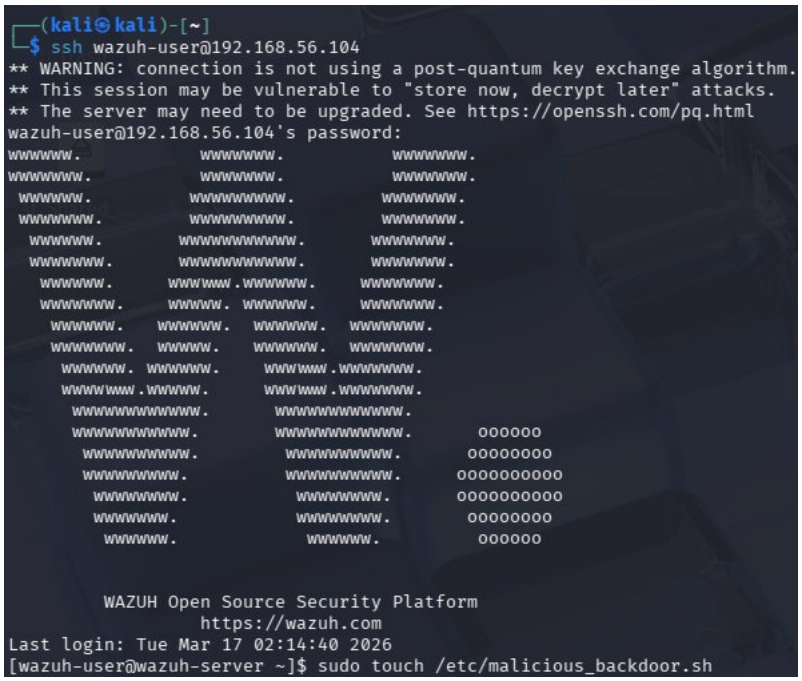
The true power of integrating a NIDS into a SIEM is the ability to correlate an attack across both the network and the host layers to build a complete narrative.

To demonstrate this, an Nmap scan was launched against the Ubuntu Server (192.168.56.104) to trigger a network reconnaissance alert.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS -p 1-1000 192.168.56.104
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-16 22:15 EDT
Nmap scan report for 192.168.56.104
Host is up (0.00040s latency).
Not shown: 998 closed tcp ports (reset)
PORT    STATE SERVICE
22/tcp  open  ssh
443/tcp open  https
MAC Address: 08:00:27:C8:AE:22 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 14.35 seconds
```

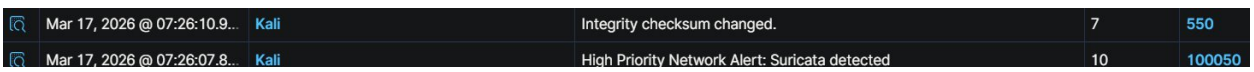
Immediately following the scan, an SSH session was established with the target server, and a malicious script (/etc/malicious_backdoor.sh) was created to trigger a Wazuh File Integrity Monitoring (FIM) alert.



The Nmap network event correctly logged the targeted behavior.



By filtering the Threat Hunting dashboard by the Ubuntu Server, both events were viewed simultaneously on a unified timeline.



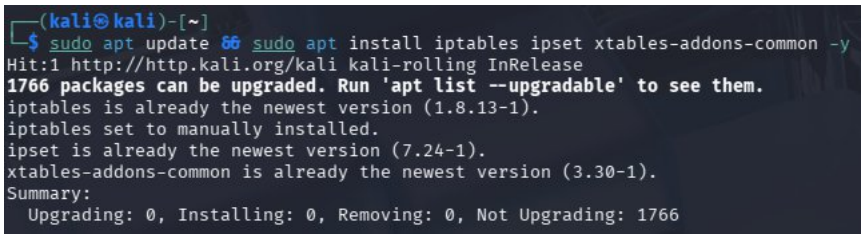
Correlation Conclusion: On its own, the Suricata alert only indicates reconnaissance; it does not confirm a breach. On its own, the Wazuh FIM alert shows a system change, but doesn't explain how the attacker identified the target. By correlating these alerts in a single pane of glass, a complete attack narrative emerges: The attacker mapped the server's open ports via a SYN scan, successfully authenticated, and established persistence by dropping a payload into a sensitive directory. Without both tools, the SOC would operate with a critical blind spot.

Days 20 and 21 — GeoIP Blocking and Firewall Rules

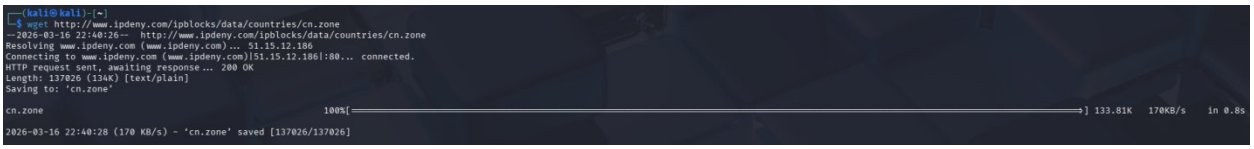
The final phase shifted from active detection to perimeter defense. GeoIP blocking was implemented to universally drop traffic originating from specific geographic regions that hold no legitimate business case for accessing the network.

Implementing GeoIP Restrictions

The required kernel-level tools (iptables, ipset, and xtables-addons-common) were installed.



A raw zone file containing over 8,000 subnets allocated to China (cn.zone) was downloaded from ipdeny.com.



A dedicated ipset bucket (block_cn) was initialized in the kernel to hold the target addresses.



A for loop was executed to iterate through the text file and push all 8,000+ CIDR blocks into the ipset. *(Note: In a production environment, ipset restore is preferred to bypass the significant time delay of iterative loop injection).*



With the bucket populated, a single iptables rule was injected at the top of the INPUT chain, instructing the firewall to actively DROP any packets originating from a source IP found within the block_cn set.



Verification of the firewall ruleset confirmed the drop logic was correctly bound to the hash:net list.

```
┌──(kali㉿kali)-[~]
└─$ sudo iptables -L INPUT -v -n
sudo ipset list block_cn | head -n 15
Chain INPUT (policy ACCEPT 12 packets, 713 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       all  --  *      *       0.0.0.0/0            0.0.0.0/0           match-set block_cn src
Name: block_cn
Type: hash:net
Revision: 7
Header: family inet hashsize 2048 maxelem 65536 bucketsize 12 initval 0×d3d2ebfd
Size in memory: 235248
References: 1
Number of entries: 8803
Members:
103.33.124.0/22
117.112.0.0/13
202.96.136.0/21
202.122.132.0/24
203.83.12.0/22
45.252.172.0/22
103.217.188.0/22
```

Local DNS Sinkholing

To compliment inbound blocking, outbound DNS restrictions were configured via the /etc/hosts file.

```
┌──(kali㉿kali)-[~]
└─$ sudo nano /etc/hosts
```

Simulated malicious domains were routed directly to the 127.0.0.1 loopback address, effectively creating a local DNS sinkhole.

```
  GNU nano 8.6
127.0.0.1       localhost
127.0.1.1       kali
::1             localhost ip6-localhost ip6-loopback
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
127.0.0.1    evil-malware-domain.com
127.0.0.1    bad-phishing-site.net
```

Testing via curl confirmed that requests to the flagged domains were instantly hijacked and met with a Connection refused error, preventing the machine from reaching the actual internet endpoints.

```
┌──(kali㉿kali)-[~]
└─$ curl -v http://evil-malware-domain.com
* Host evil-malware-domain.com:80 was resolved.
* IPv6: (none)
* IPv4: 127.0.0.1
*   Trying 127.0.0.1:80 ...
* connect to 127.0.0.1 port 80 from 127.0.0.1 port 48260 failed: Connection refused
* Failed to connect to evil-malware-domain.com port 80 after 12 ms: Could not connect to server
* closing connection #0
curl: (7) Failed to connect to evil-malware-domain.com port 80 after 12 ms: Could not connect to server
```

Security Limitations and Wazuh Visibility

Concept	Analysis
IPSet vs. IPtables	ipset is utilized because it stores thousands of IP ranges in a highly optimized hash map. If iptables had to read 8,000 individual rules line-by-line for every packet, network performance would collapse. ipset allows the firewall to check one single rule instantly.
DROP vs. REJECT	The firewall was configured to DROP traffic, which silently discards the packet, forcing the attacker's scanner to time out and wasting their resources. A REJECT rule actively sends an ICMP "Connection Refused" packet back, which confirms to the attacker that the host is alive and actively blocking them.
Wazuh Visibility	Wazuh did not generate an alert for this blocked traffic. A standard iptables -j DROP rule discards the

	packet silently without writing an entry to syslog or kern.log. Because no local log is generated, the Wazuh agent has nothing to report. To achieve SIEM visibility, a dedicated -j LOG rule must be placed directly above the DROP rule.
GeoIP Limitations & Persistence	GeoIP blocking is merely a layer of friction. Sophisticated threat actors easily bypass it by routing attacks through compromised infrastructure or VPNs located in permitted countries. Furthermore, these iptables and ipset rules reside in volatile memory; they will vanish upon system reboot unless explicitly saved and restored using a firewall manager like ufw or iptables-persistent.

End of Week 3 Report